

角色分开以后，前一步的工作怎样接着用？

问题不在角色数量，而在资料、中间状态和处理方法能否接续。

一次交接，三处可能重做

通用协作场景示意

01

交接时

资料写进长消息

下游再找一次所需数据行
来源位置还要重新核对

02

处理中

选择结果只剩文字

“选中了第 3 行”
却没有带回原文位置

03

下一次

换一批输入又重来

相同的汇总方法
下一轮重新生成一次

对应赛题：交接要有结构，状态要能消费，方法要能跨任务复用。

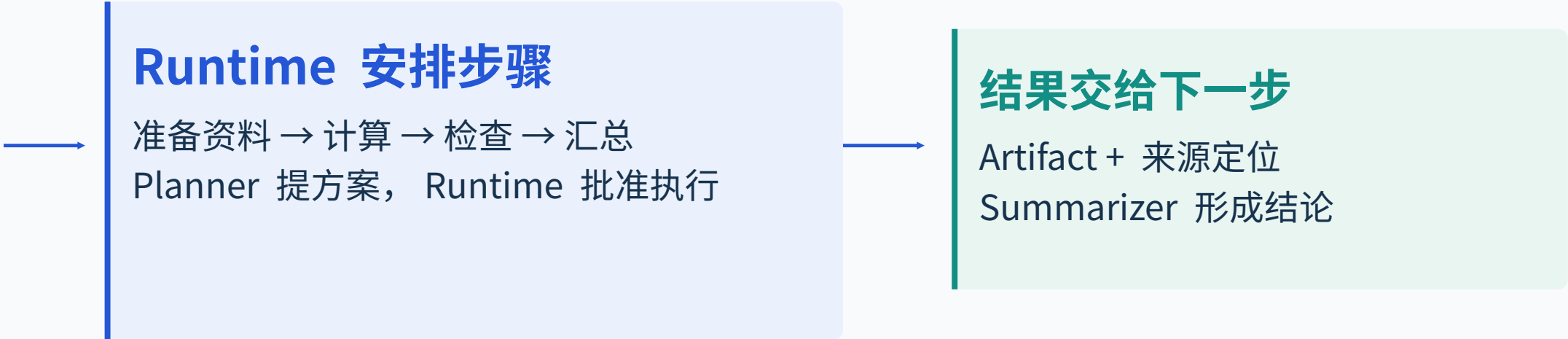
StateBus 把交接变成 “步骤 + 对象 + 方法”

Runtime 安排执行；协议说清请求；Ref 定位资料 and 结果；Memory 留下可复用方法。

贯穿例：月度核算

2 月各业务单元
收入、成本、利润

输入：本月 CSV
输出：结构化结果 + 引用



协议

这次做什么、读什么、交什么

引用对象

资料和计算结果都有明确位置

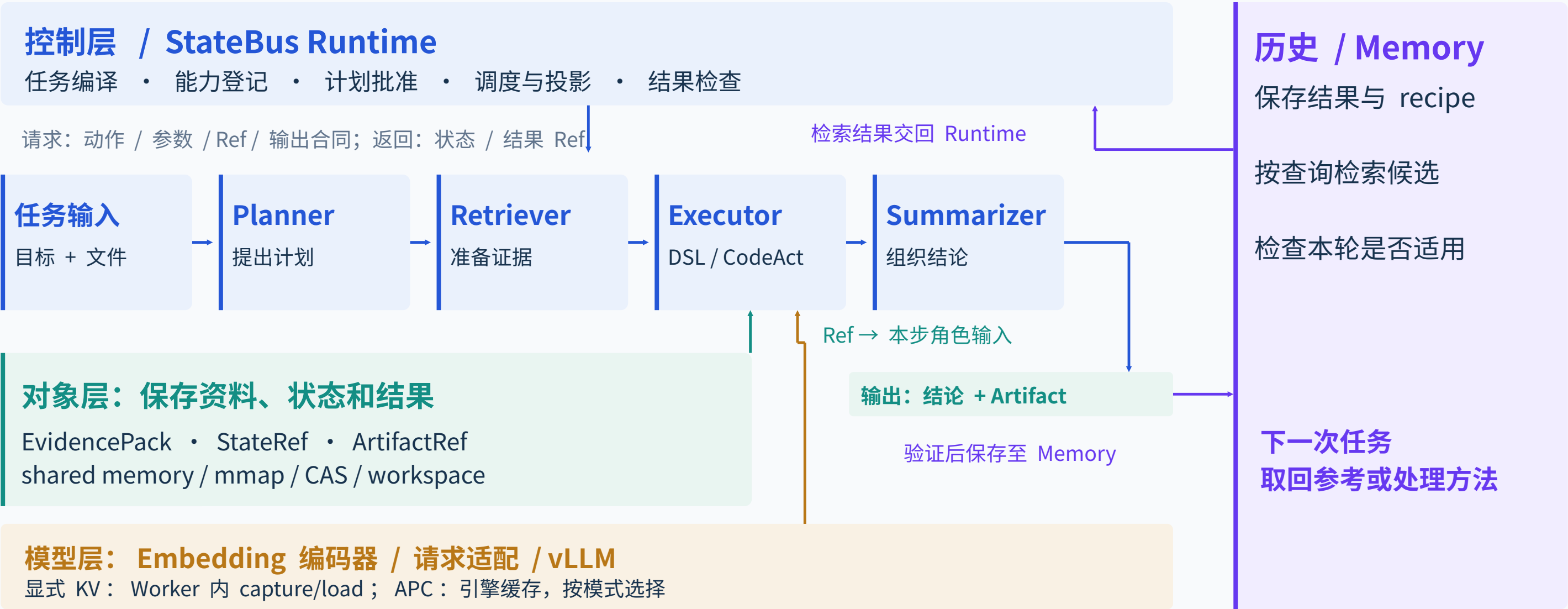
Memory

上月方法可用于本月重新计算

F01 汇总 1 月，F02 汇总 2 月：可以复用方法，仍需读取并计算 2 月数据。

一次任务从输入到结论，由 Runtime 推进

横向读任务主线；对象、模型服务与历史记忆在各自位置接入。



四个协作问题，分别在不同位置解决

亮点按问题展开； Embedding、Logit、显式 KV 和 APC 各有使用条件。

01

交接时传什么？
能力、请求、结果引用

结构化协议

02

中间工作怎样继续用？
选证据 / 判断展开 / 复用计算

Embedding • Logit • KV • APC

03

拿到资料怎样算完？
固定变换或受限 Python，交付同一种结果

DSL / CodeAct

04

下一轮怎样少重做？
找到历史方法，在当前输入上重新执行

Memory

协议规定交接；状态保留中间工作；执行产出结果；Memory 让方法跨任务复用。

一次交接先约定输入和输出，再交给 Worker

协议让接收方知道要读哪个对象、执行什么动作、按什么要求交付。



投影： EvidenceRef → 本步数据行 + locator；下游不再从长消息里猜输入。

跨进程控制消息走 Protobuf / UDS；对象内容由 Ref 指向共享内存、mmap 或 workspace。

向量选出候选行， manifest 把它还原成证据

矩阵负责排序； Executor 最终拿到的是可计算、可追溯的证据。

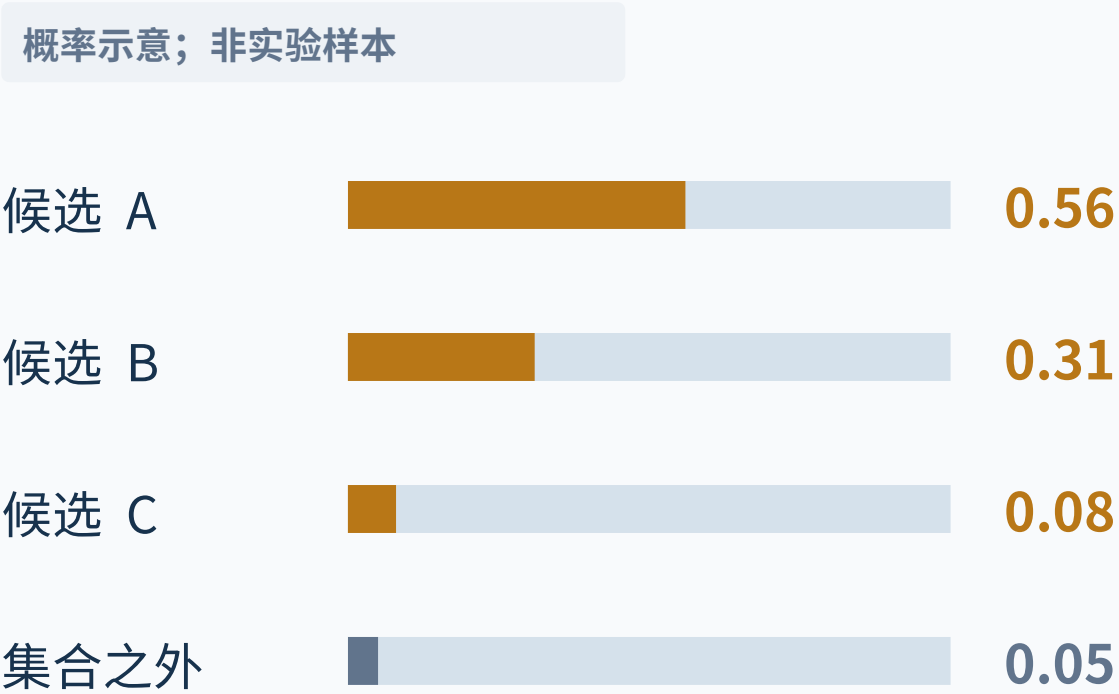


控制消息只带引用与选择参数；矩阵通过对象载体读取。

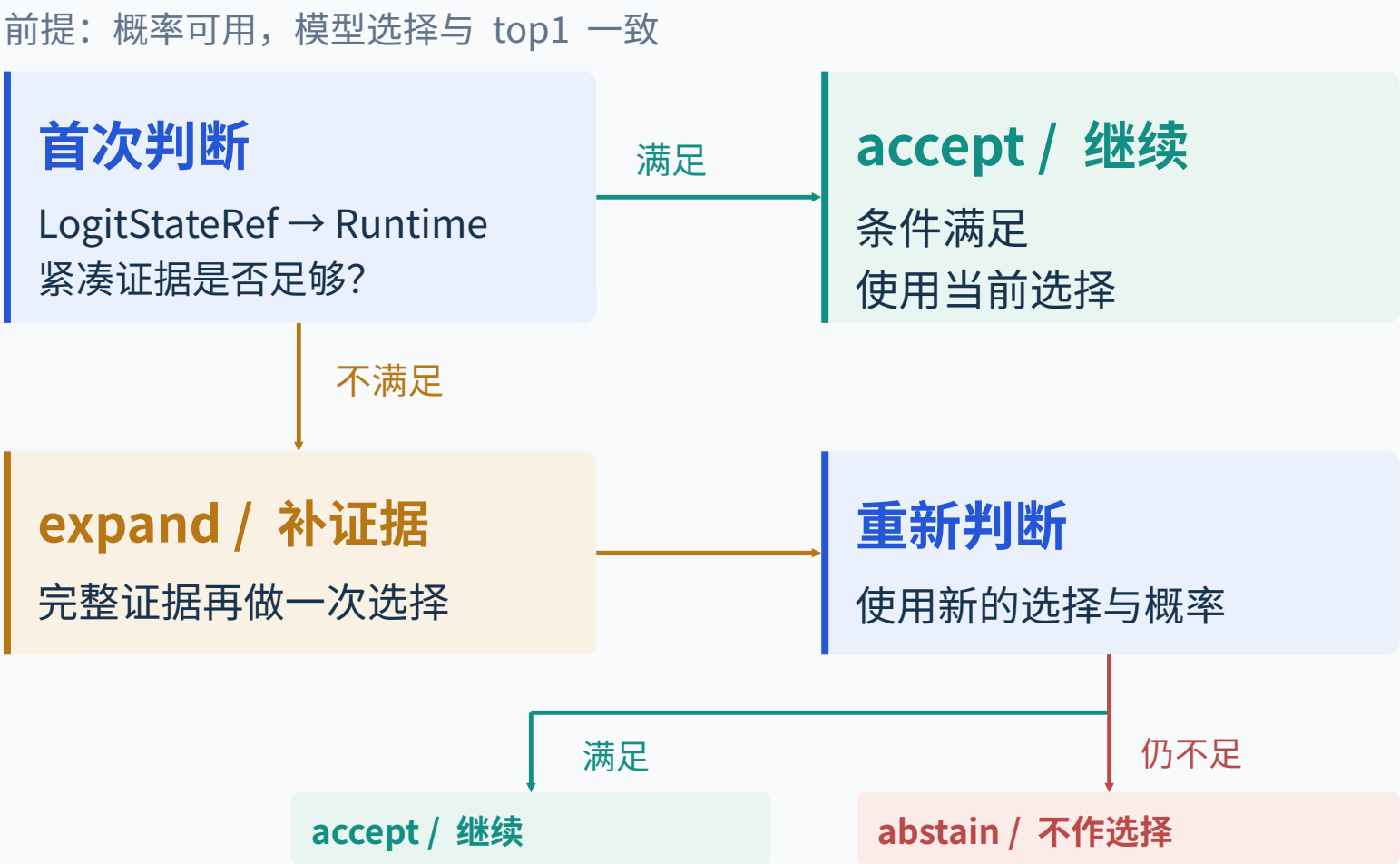
状态生命周期： `publish -> transfer -> consume -> release` ； 矩阵更小不是本页的收益前提。

先看紧凑证据，拿不准时才追加材料

margin 看前两名差距， other_mass 看候选集合之外的概率；两者一起决定是否展开。



margin = p_top1 - p_top2 = 0.25
other_mass = 1 - Σ p_i = 0.05

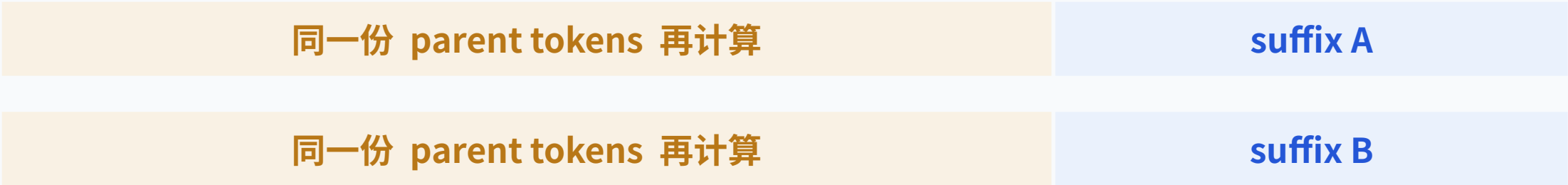


专项门槛： tau = 0.10 ， other_mass_limit = 0.20 ； 概率用于控制取证，不保证答案正确。

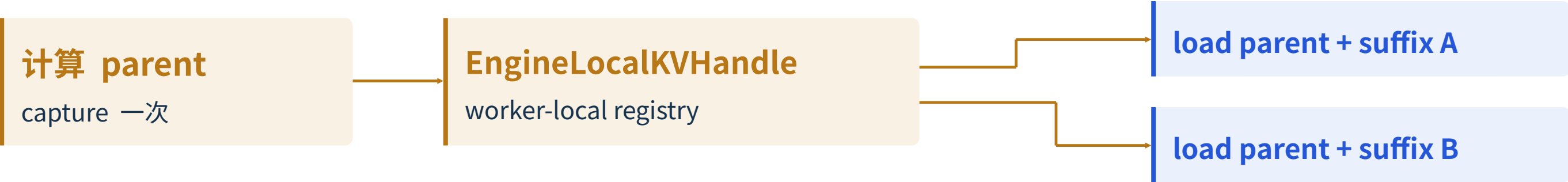
公共 parent 只计算一次，后续请求接着算

显式 KV 在同一 vLLM Worker 内通过 handle 继续使用 parent ； 不跨模型搬运张量。

重复 full prefill



同一 vLLM Worker / continuation



省去重复 parent prefill ； capture / load 仍有成本。显式 KV 模式不与 APC 同时开启。

复用 F01 的方法，重新计算 F02 的 2 月数据

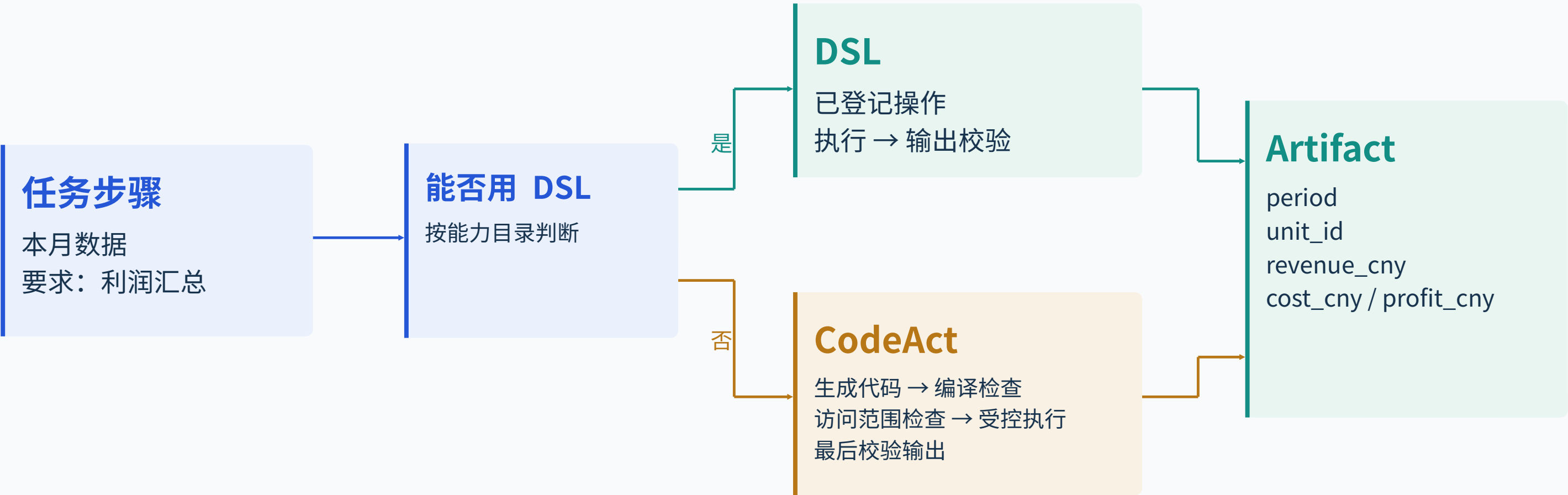
历史记录提供 recipe ； 当前数据和新结果仍在本轮读取、执行和验证。



F02 实测：复用来源为 F01 ； 本轮输入执行已记录； Executor 模型请求从 1 次降为 0 次。

DSL 与 CodeAct 两条路径，交付同一类结果

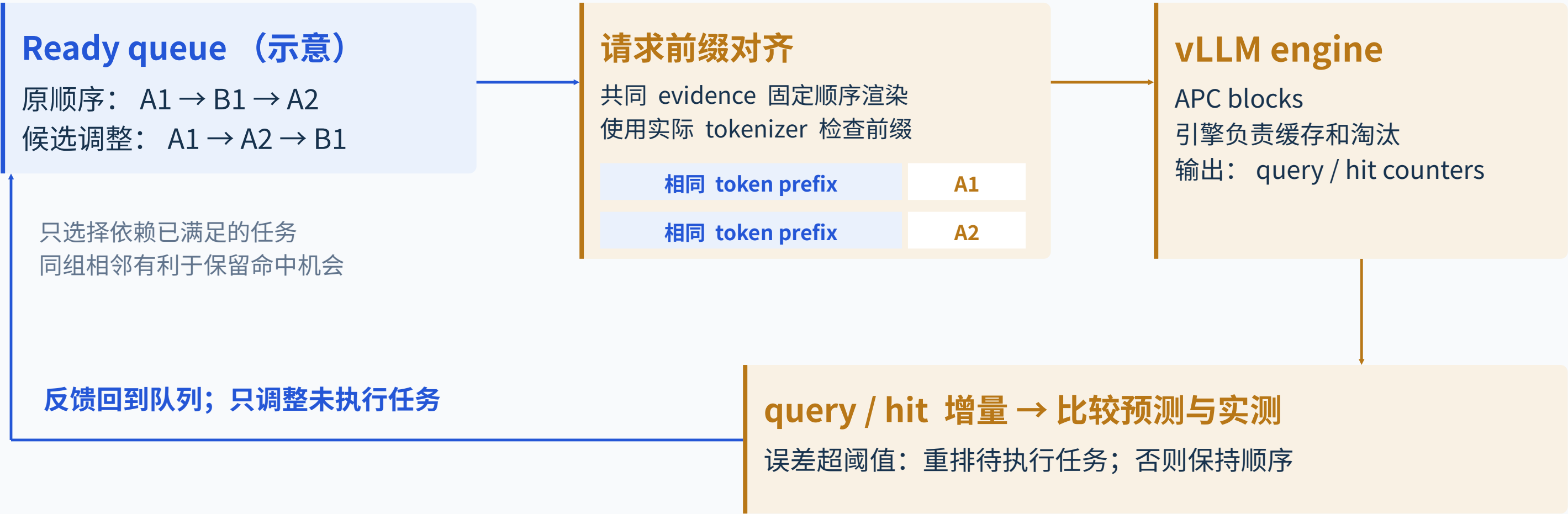
已定义变换走 DSL ；超出表达范围时，受限 Python 完成计算并接受输出检查。



下游读取结构化产物和来源；执行方式改变，要求的字段、类型和质量门不变。

StateBus 对齐公共前缀， vLLM 决定实际命中

相同资料要变成相同 token 前缀；命中反馈用于调整尚未执行的 ready task。



Runtime 改请求布局与顺序；缓存 block 留在 vLLM 内。命中增加不等于整项任务同比例加速。

同一份资料按角色裁剪，来源始终保留

角色视图示意 + F02 实测样例：原始输入、计算结果和引用分别展示。



F02 / U-A 原始 CSV 第 1 条记录

net_revenue_cny=104,908 • cost_cny=79,730
来源：finance/actual_2026-02.csv（首条数据）

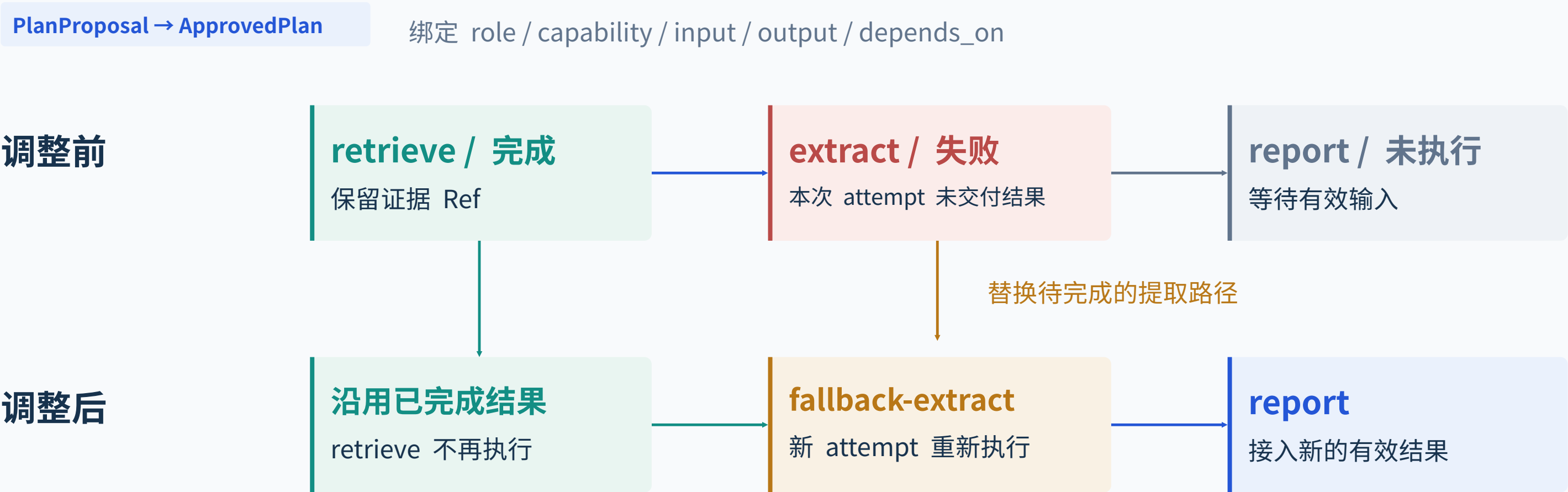
U-A 共 6 行汇总 → 新 Artifact

收入 625,229 – 成本 475,172 = 利润 150,057 元
period=2026-02；文件 lineage 随产物保留

输入行不是计算结果：Executor 汇总本月数据，Summarizer 读取新产物并引用来源。

计划变化时，保留已完成结果，只调整后续步骤

Planner 提方案； Runtime 绑定真实能力、输入输出要求和依赖，再批准执行。



保留的是已完成步骤的有效结果；替换和重试受能力目录、依赖与预算约束。

历史方法是否可用，要和本轮条件逐项比较

输入内容可以变化；方法要求的字段、版本兼容性和输出合同必须成立。

F02 实测：F01 方法 × 本月输入

检查项	历史方法要求	本轮必须确认
输入内容	F01： 1 月数据及来源	F02： 2 月数据，来源已变
执行方法	Executor / transform_dsl	同类任务合同，获准重算
输出合同	period / unit_id / 金额列	本轮仍要求相同字段与类型

F02 记录：input_lineage_changed → degraded
获准复用方法； current_input_execution_observed = true

ASSIST

可参考；仍自行完成计算

VALIDATED_REPLAY

用本轮输入重算 recipe
产生并验证新 Artifact

REJECT

条件不兼容；不使用候选

三种去向互斥：检索排名只决定先看谁，兼容性和策略决定怎么用。

两类连续任务，按相同输入和质量门配对比较

finance 与 service_ops 各 12 轮；24 对任务，两种配置共 48 次执行。

任务不是 24 个孤立问答

finance



收入 / 退款 / 成本 CSV → 按业务单元输出收入、成本和利润

service_ops



站点 hourly CSV → 请求数、最终失败数；后续连接计划与历史结果

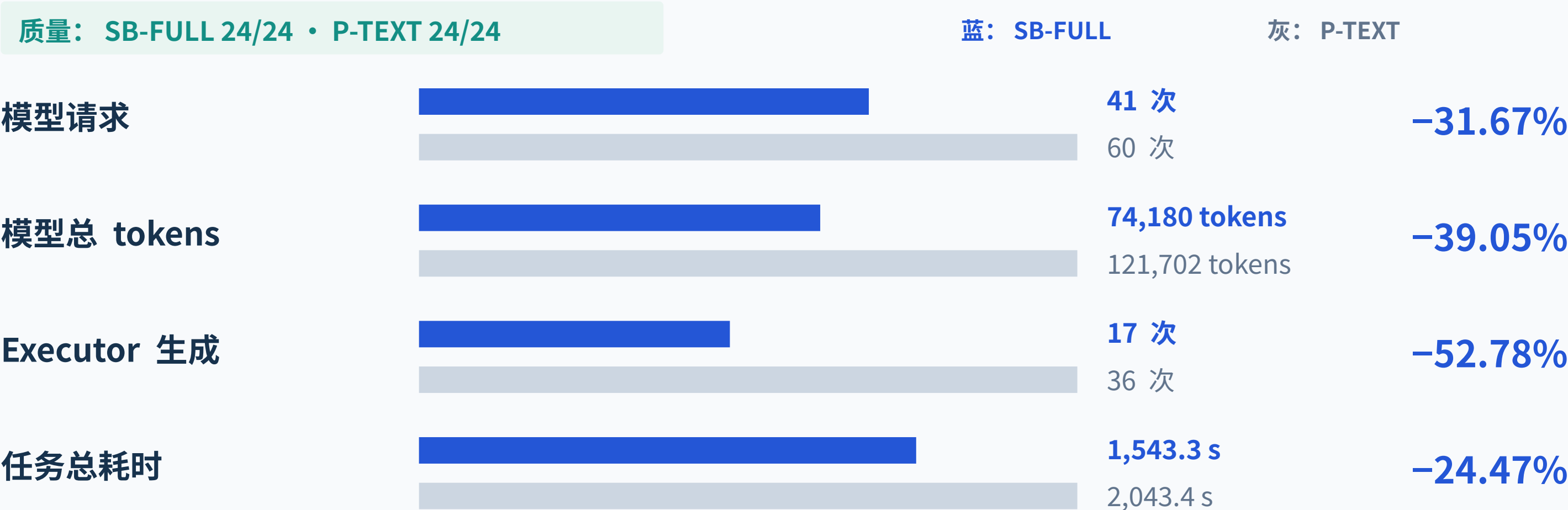
SB-FULL : typed object + State / Memory
P-TEXT : JSON text , State / Memory 关闭

相同文件、任务顺序、输出合同与质量门
openEuler 24.03 LTS-SP3 • Qwen3-32B / vLLM

采集：结果质量与耗时、模型请求与 tokens、对象交接及 State / Memory 事件。

同样通过 24 项质量门，整体开销下降

四项指标分别比较； 24 个任务耗时为逐任务 E2E 总和。



同一批任务总体比较：请求 - 31.67% ， tokens -39.05% ， 生成 - 52.78% ， 耗时 - 24.47% 。

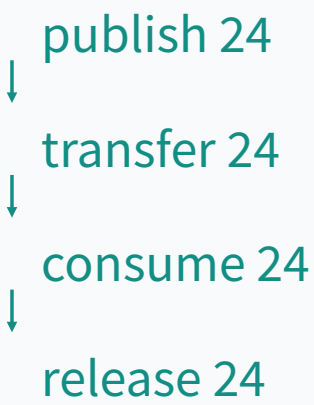
主线事件与专项记录，分别证明机制发生过

主线 24 对、Memory 专项 8 对、执行 showcase 5 例：三个分母不合并。

01 主线：交接与消费

89 条 typed / 108 条 text

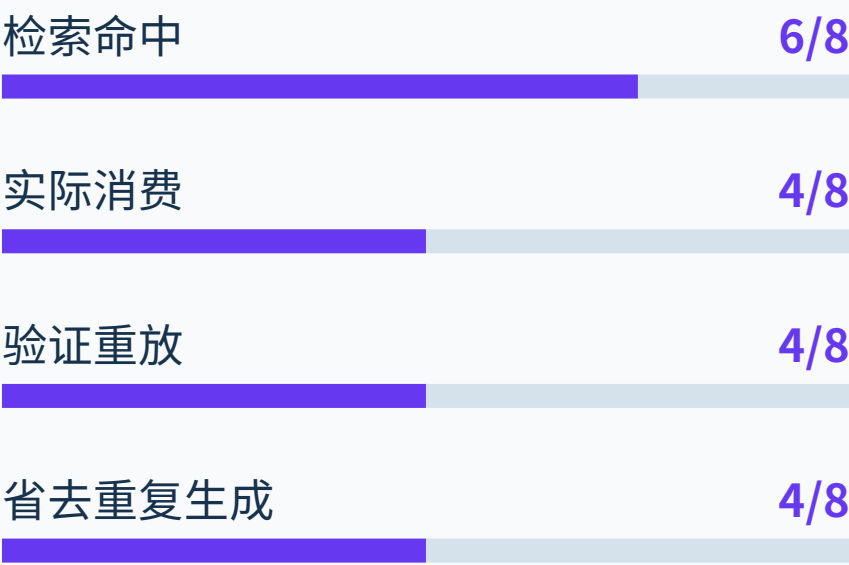
State / SB-FULL 24 次执行



Memory：24 查 → 22 次有候选
共 132 个候选，12 次实际消费

02 Memory 专项

8 个 on + 8 个 off



仍执行 recipe 并验证新结果

03 执行 showcase

DSL

2 / 3 通过

CodeAct fallback

2 / 2 通过

独立的 5 个样例
不能当作主线成功率

F02 记录：历史方法来自 F01；当前输入确实执行；减少的是重复生成请求。

四组专项：机制被使用，局部收益各有边界

每组独立任务、开关和分母；下列局部指标不与主线总耗时合并。

Embedding / State



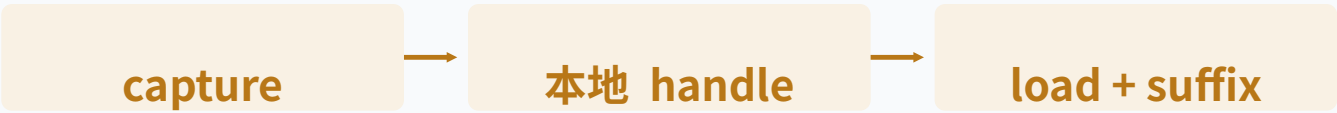
4 个任务 × off/on = 8 次；on 为 4 次
发布 / 传递 / 消费各 10 次；释放 4/4
1/4 任务观察到下游选择改变

Logit



4 个 case × 3 种模式 = 12 次执行
3 个 resolved：logical input -75.21%
compact/selective 对 full；另 1 例展开后弃答

显式 KV



4 对 / 8 次：full replay vs continuation
prefill -90.37% / 完整任务耗时 - 3.58%

APC

平均 hit tokens：48 → 5,168



4 对 / 8 次：independent vs shared prefix
TTFT -89.59% / 完整任务耗时 - 4.29%

State 证明状态被消费；Logit 控制取证预算；KV / APC 复用计算，但整项任务收益较小。